Aplicação do Problema da Mochila em Python via Métodos de Otimização Numérica

João Vitor C. Pinheiro[1],Breno Cauã R. da silva[2], Mário Diego R. Valente[3]

[1]Faculdade (Faci Wyden) – Belém – PA – Brazil

[2]Instituto de Ciências Exatas e Naturais – Universidade Federal do Pará
Faculdade de Estatística - Belém – PA – Brazil

[3]Instituto de Ciências Exatas e Naturais – Universidade Federal do Pará
Faculdade de Computação - Belém – PA – Brazil

Abstract. *This work presents a study on the Knapsack Problem and its variations: 0-1 Knapsack, Fractional Knapsack, and Multiple Knapsack. It explores algorithmic solutions, especially the Greedy Algorithm and the Classical Integer Linear Programming method using Python-MIP. Through computational implementations and simulated data, the study compares both approaches in terms of efficiency, solution quality, and computational cost. Results demonstrate the contexts where each method is most suitable, reinforcing the importance of algorithmic choice in resource allocation and optimization scenarios.*

Resumo. *Este trabalho apresenta um estudo sobre o Problema da Mochila e suas variações: Mochila 0-1, Mochila Fracionária e Mochila Múltipla. São exploradas soluções algorítmicas, com foco no Algoritmo Guloso e no Método Clássico de Programação Linear Inteira via Python-MIP. Por meio de implementações computacionais e dados simulados, o estudo compara as abordagens quanto à eficiência, qualidade das soluções e custo computacional. Os resultados demonstram em quais contextos cada método é mais adequado, reforçando a importância da escolha algorítmica em cenários de alocação de recursos e otimização.*

1. Introdução

O Problema da Mochila é um dos problemas clássicos da otimização combinatória e consiste em selecionar um subconjunto de itens, cada um com peso e valor associados, de forma a maximizar o valor total sem exceder uma capacidade limitada. Apesar de sua formulação simples, trata-se de um problema computacionalmente desafiador e amplamente estudado na literatura de Pesquisa Operacional e Ciência da Computação.

Esse problema surge naturalmente em diversas aplicações reais, como planejamento financeiro, gerenciamento de estoques, alocação de recursos e otimização de processos logísticos. Em tais contextos, a tomada de decisão envolve a escolha eficiente entre alternativas limitadas por restrições de capacidade, tornando o estudo do Problema da Mochila altamente relevante para sistemas de apoio à decisão.

Ao longo das últimas décadas, diferentes abordagens têm sido propostas para sua resolução, incluindo métodos exatos, baseados em programação matemática e programação dinâmica, e métodos heurísticos, que buscam soluções aproximadas com menor custo computacional. A escolha da abordagem depende do tamanho da instância e do nível de precisão desejado [Martello e Toth 1990].

Diante desse cenário, este trabalho tem como objetivo apresentar os principais conceitos relacionados ao Problema da Mochila, discutir suas variações e analisar diferentes estratégias de solução, com ênfase na comparação entre abordagens exatas e heurísticas.

2. O Problema da Mochila

O problema da mochila busca maximizar o valor total dos itens selecionados sem ultrapassar a capacidade disponível. Esse problema possui diversas variações, cada demanda de abordagens distintas para a obtenção de soluções eficientes.

2.1. Mochila 0-1

O Problema da Mochila 0-1 é um clássico problema de otimização combinatória em que, dada uma mochila com capacidade máxima W e um conjunto de n itens — cada um com peso w_i e valor v_i — busca-se selecionar um subconjunto de itens que maximize o valor total sem exceder a capacidade disponível.

$$\begin{aligned} \text{Maximizar:} \quad & \sum_{i=1}^{n} v_i x_i \\ \text{Sujeito a:} \quad & \sum_{i=1}^{n} w_i x_i \leq W \\ \text{Onde:} \quad & x_i \in \{0, 1\},\ \forall i = 1, 2, \ldots, n \end{aligned}$$

Nesse modelo, cada variável binária x_i indica se o item i é selecionado ($x_i = 1$) ou não ($x_i = 0$). O problema é classificado como NP-difícil, o que implica que não há, até o momento, algoritmos conhecidos capazes de resolvê-lo de forma exata em tempo

polinomial para todas as instâncias. Entre as principais abordagens de solução destacam-se a programação dinâmica, métodos de enumeração completa e técnicas de programação inteira.

Para ilustrar o funcionamento do problema, considere o conjunto de itens apresentado na Tabela 1 e uma mochila com capacidade $W = 5$.

Tabela 1. Exemplo de itens.

Item	**Peso** (w_i)	**Valor** (v_i)
1	2	3
2	3	4
3	4	5

As possíveis combinações de itens são:

- Itens $\{1\} \rightarrow$ Peso: 2, Valor: 3
- Itens $\{2\} \rightarrow$ Peso: 3, Valor: 4
- Itens $\{3\} \rightarrow$ Peso: 4, Valor: 5
- Itens $\{1, 2\} \rightarrow$ Peso: 5, Valor: 7 **(Solução ótima)**
- Itens $\{1, 3\} \rightarrow$ Peso: 6 (inviável)
- Itens $\{2, 3\} \rightarrow$ Peso: 7 (inviável)
- Itens $\{1, 2, 3\} \rightarrow$ Peso: 9 (inviável)

Observa-se que a melhor solução consiste na seleção dos itens 1 e 2, que atingem exatamente a capacidade da mochila e produzem o valor máximo de 7. Embora simples, esse exemplo evidencia a natureza combinatória do problema e a necessidade de métodos computacionais eficientes para resolver instâncias de maior escala.

2.2. Mochila Fracionária

Na variação fracionária do Problema da Mochila, admite-se que os itens possam ser divididos, permitindo a inclusão de frações de cada objeto na mochila. Assim, cada item pode ser selecionado parcialmente, desde que a soma total dos pesos não ultrapasse a capacidade disponível.

A formulação matemática desse problema é dada por:

$$\begin{aligned} \text{Maximizar:} \quad & \sum_{i=1}^{n} v_i x_i \\ \text{Sujeito a:} \quad & \sum_{i=1}^{n} w_i x_i \leq W \\ \text{Onde:} \quad & 0 \leq x_i \leq 1, \, \forall i = 1, 2, \ldots, n \end{aligned}$$

Diferentemente da mochila 0-1, essa versão pode ser resolvida de forma ótima por meio de um algoritmo guloso. O método consiste em ordenar os itens de acordo com a razão valor/peso (v_i/w_i) e inseri-los na mochila em ordem decrescente dessa razão, adicionando frações do último item quando necessário para preencher a capacidade restante.

Essa abordagem garante a obtenção da solução ótima em tempo polinomial, uma vez que a possibilidade de fracionamento elimina a natureza combinatória presente na versão 0-1 do problema. Como resultado, a mochila fracionária representa um caso particular mais simples, frequentemente utilizado como base para a compreensão de métodos exploratórios e relaxações do problema original.

2.3. Problema das Múltiplas Mochilas

O Problema das Múltiplas Mochilas (Multiple Knapsack Problem — MKP) é uma generalização do Problema da Mochila 0-1, na qual múltiplas mochilas com capacidades distintas estão disponíveis. O objetivo consiste em alocar um conjunto de itens, cada um com peso e valor associados, entre as mochilas de modo a maximizar o valor total transportado sem violar as restrições de capacidade.

Assim como a versão clássica, o MKP é classificado como um problema NP-difícil, herdando a natureza combinatória do problema da mochila 0-1. Essa variação possui diversas aplicações práticas, incluindo logística, alocação de recursos, escalonamento e planejamento de produção.

2.3.1. A formulação matemática do problema pode ser descrita a seguir.

Sejam I o conjunto de itens e M o conjunto de mochilas. Cada item $i \in I$ possui peso P_i e valor V_i, enquanto cada mochila $j \in M$ possui capacidade W_j. Define-se a variável binária:

$$X_{ij} = \begin{cases} 1, & \text{se o item } i \text{ é alocado à mochila } j \\ 0, & \text{caso contrário} \end{cases}$$

O modelo de otimização é dado por:

$$\text{Maximizar} \sum_{j \in M} \sum_{i \in I} V_i X_{ij}$$

sujeito a:

$$\sum_{i \in I} P_i X_{ij} \leq W_j, \quad \forall j \in M$$

$$\sum_{j \in M} X_{ij} \leq 1, \quad \forall i \in I$$

$$X_{ij} \in \{0, 1\}, \quad \forall i \in I, \forall j \in M$$

2.3.2. Exemplo Prático: Otimização da Coleta Seletiva

Contexto:

Para ilustrar a aplicação prática do modelo, considera-se um cenário de otimização da coleta seletiva.

Objetivo:

Esse exemplo ilustra como o MKP pode ser utilizado na modelagem de problemas reais de distribuição e logística. A formulação permite representar explicitamente as restrições de capacidade e exclusividade de alocação, possibilitando a aplicação de métodos de otimização para a obtenção de soluções eficientes.

Dados

Tabela 2. Dados dos resíduos, seus respectivos pesos e valores.

Item i	**Tipo de Resíduo**	**Peso** P_i **(kg)**	**Valor** V_i
1	Plástico	10	8
2	Papel	15	10
3	Vidro	12	6
4	Metal	18	11
5	Eletrônico	20	14
6	Orgânico compostável	8	5

Tabela 3. Dados dos caminhões e suas respectivas capacidades.

Caminhão j	**Capacidade** W_j **(kg)**
1	30
2	35
3	25

Formulação para o Exemplo:

Função Objetivo:

$$\text{Maximizar} \sum_{j=1}^{3} \sum_{i=1}^{6} V_i X_{ij}$$

Restrições:

1. Capacidade dos caminhões:

$$\sum_{i=1}^{6} P_i X_{ij} \leq W_j, \quad j = 1, 2, 3$$

2. Cada resíduo em no máximo um caminhão:

$$\sum_{j=1}^{3} X_{ij} \leq 1, \quad i = 1, 2, \ldots, 6$$

Domínio:

$$X_{ij} \in \{0, 1\}$$

2.3.3. Interpretação do modelo

A formulação apresentada permite representar de forma estruturada problemas reais de alocação de recursos com múltiplas restrições de capacidade. As variáveis binárias indicam explicitamente a decisão de alocação de cada item a uma mochila específica, enquanto as restrições garantem simultaneamente a viabilidade das soluções e a exclusividade de alocação.

Essa modelagem possibilita a aplicação de técnicas de otimização inteira para a obtenção de soluções ótimas ou aproximadas, tornando o Problema das Múltiplas Mochilas particularmente relevante em contextos logísticos e industriais. Além disso, a estrutura do modelo facilita sua adaptação para cenários mais complexos, como inclusão de custos, prioridades ou múltiplos critérios de decisão.

2.4. Algoritmo Guloso

O algoritmo guloso é uma estratégia de otimização baseada na escolha iterativa da melhor decisão local na expectativa de alcançar uma solução globalmente ótima. Essa abordagem é particularmente eficiente quando o problema apresenta propriedades que garantem a validade dessas decisões locais.

No contexto do Problema da Mochila Fracionária, o método guloso produz a solução ótima em tempo polinomial. A estratégia consiste em ordenar os itens de acordo com a razão valor/peso (v_i/w_i) e inseri-los na mochila em ordem decrescente dessa razão até que a capacidade seja atingida. Caso o último item não caiba integralmente, adiciona-se apenas a fração necessária para preencher a capacidade restante.

O procedimento pode ser resumido nos seguintes passos:

1. Calcular a razão valor/peso para cada item.
2. Ordenar os itens em ordem decrescente dessa razão.
3. Inserir os itens sequencialmente na mochila até atingir a capacidade.
4. Se necessário, inserir uma fração do último item.

A complexidade do algoritmo é dominada pela etapa de ordenação, resultando em tempo $O(n \log n)$.

Entretanto, essa abordagem não garante a solução ótima para o Problema da Mochila 0-1. A impossibilidade de fracionamento impede que decisões locais baseadas apenas na razão valor/peso conduzam necessariamente à solução global ótima, exigindo a utilização de métodos exatos ou heurísticos mais sofisticados.

Dessa forma, observa-se a seguinte distinção fundamental:

- Mochila 0-1: o algoritmo guloso não garante optimalidade.
- Mochila Fracionária: o algoritmo guloso produz a solução ótima.

2.5. Análise Comparativa da Aplicação do Algoritmo Guloso nas Versões 0-1 e Fracionária da Mochila

2.5.1. Mochila 0-1: Quando o Algoritmo Guloso Falha

Problema:

Capacidade da mochila: $W = 50$

Tabela 4. Dados dos itens com seus respectivos valores, pesos e razão valor/-peso.

Item	**Valor** (v_i)	**Peso** (w_i)	**Valor/Peso** (v_i/w_i)
1	60	10	6.0
2	100	20	5.0
3	120	30	4.0

Aplicando o algoritmo guloso (baseado em valor/peso):

- Escolhe o item 1 $\rightarrow$ peso acumulado = 10, valor = 60
- Escolhe o item 2 $\rightarrow$ peso acumulado = 30, valor = 160
- Item 3 não cabe

Valor total obtido: 160

Solução ótima (via força bruta ou programação dinâmica):

- Escolhe item 2 e item 3 $\rightarrow$ peso total = 50, valor = $100 + 120 = 220$

Observa-se que o algoritmo guloso não produz a solução ótima para a mochila 0-1. Isso ocorre porque a estratégia gulosa baseia-se exclusivamente em decisões locais, ignorando combinações futuras de itens que poderiam gerar maior valor global. A ausência da propriedade de escolha gulosa (greedy-choice property) impede que a seleção baseada apenas na razão valor/peso conduza necessariamente à solução ótima nesse contexto discreto.

2.5.2. Mochila Fracionária: Quando o Algoritmo Guloso Funciona

Considerando os mesmos dados e capacidade $W = 50$, aplica-se o algoritmo guloso fracionário:

- Os itens são ordenados pela razão v_i/w_i: item 1 (6.0), item 2 (5.0) e item 3 (4.0);
- O item 1 é inserido integralmente, resultando em valor acumulado de 60;
- O item 2 é inserido integralmente, elevando o valor acumulado para 160;
- A capacidade restante permite a inclusão de $\frac{2}{3}$ do item 3, adicionando valor 80.

Valor total: $60 + 100 + 80 = 240$

Fórmula da mochila fracionária:

$$\max \sum_{i=1}^{n} v_i \cdot x_i \quad \text{sujeito a} \quad \sum_{i=1}^{n} w_i \cdot x_i \leq 50, \quad 0 \leq x_i \leq 1$$

Com:

$$x_1 = 1, \quad x_2 = 1, \quad x_3 = \frac{2}{3}$$

$$\text{Valor total} = 60 \cdot 1 + 100 \cdot 1 + 120 \cdot \frac{2}{3} = 240$$

2.5.3. Resumo Comparativo

Os resultados obtidos evidenciam a diferença fundamental entre as duas variações do problema. Enquanto a natureza discreta da mochila 0-1 impede a aplicação eficaz da estratégia gulosa, a possibilidade de fracionamento garante a validade das decisões locais no caso fracionário.

Tabela 5. Comparação entre os tipos de mochila, possibilidade de fracionamento, desempenho do algoritmo guloso e valor obtido no exemplo.

Tipo de Mochila	Pode fracionar?	Guloso é ótimo?	Valor obtido (exemplo)
Mochila 0-1	Não	Não	160
Mochila Fracionária	Sim	Sim	240

3. Solução Computacional para o Problema das Múltiplas Mochilas

Após compreendermos as variações e formulações do Problema da Mochila, é fundamental analisar como ele pode ser resolvido na prática, especialmente com o apoio da computação. Existem diferentes métodos que podem ser aplicados para encontrar uma solução, variando desde abordagens exatas, que garantem o melhor resultado possível, até métodos aproximados, mais indicados para situações onde o tempo de execução é um fator crítico. A seguir, serão apresentados os principais métodos de solução utilizados na computação para resolver esse problema, com destaque para suas características, funcionamento e aplicações mais comuns.

3.1. Introdução

O problema das múltiplas mochilas envolve a alocação de itens em várias mochilas com capacidades limitadas, visando maximizar o valor total transportado. Este trabalho apresenta uma solução computacional para o desenvolvida em Python, utilizando a biblioteca `Python-MIP` (Túlio Toffolo e Haroldo Gambini, 2025) para modelagem e resolução do problema como um programa de programação linear inteira .

A implementação abrange desde a geração automatizada dos dados de entrada, como pesos dos itens, valores associados e capacidades das mochilas, até a construção do

modelo matemático, sua otimização e a visualização dos resultados obtidos. A abordagem adotada permite não apenas resolver eficientemente o problema, mas também analisar a distribuição ótima dos itens, fornecendo insights sobre o processo de alocação. O código desenvolvido será detalhado a seguir, demonstrando cada etapa do processo de forma clara e reproduzível.

3.2. Geração dos Dados

Inicialmente, define-se o número de produtos (15) e de mochilas (6), além de uma semente aleatória para garantir a reprodutibilidade dos dados. Os produtos são criados com valores aleatórios entre R$10 e R$150, e pesos entre 100g e 250g. As mochilas, por sua vez, têm capacidades máximas entre 1500g e 5000g.

```python
from mip import *
import random

RANDOM_SEED = 120
NUM_PRODUTOS = 15
NUM_MOCHILAS = 6
random.seed(RANDOM_SEED)

produtos = {
    f'p_{i}': {'valor': random.randint(10, 150),
               'peso': random.randint(100, 250)}
    for i in range(NUM_PRODUTOS)
}

mochilas = {
    f'm_{i}': {'carga_maxima': random.randint(1500, 5000)}
    for i in range(NUM_MOCHILAS)
}
```

Listing 1. Geração dos dados

3.3. Modelagem Matemática

Utilizou-se o pacote `MIP` (Túlio Toffolo e Haroldo Gambini, 2025) para construir o modelo de otimização. O pacote fornece ferramentas para modelagem e resolução de problemas de programação linear interira mista.

Cada par (mochila, produto) é representado por uma variável binária que indica se um item foi ou não colocado naquela mochila.

```python
modelo = Model(sense=MAXIMIZE)

carga = {
    (m, p): modelo.add_var(var_type=BINARY)
    for m in mochilas
    for p in produtos
}
```

Listing 2. Definição das variáveis

As principais restrições são:

- **Restrição de unicidade:** Cada produto pode ser alocado em, no máximo, uma mochila.
- **Restrição de capacidade:** O peso total dos produtos em uma mochila não pode ultrapassar sua capacidade.

```
for p in produtos:
    modelo += xsum(carga[m, p] for m in mochilas) <= 1

for m in mochilas:
    modelo += xsum(carga[m, p] * produtos[p]['peso']
                   for p in produtos) <= mochilas[m]['
                      carga_maxima']
```

Listing 3. Restrições do modelo

Função Objetivo: Maximizar o valor total dos produtos alocados.

```
modelo.objective = maximize(
    xsum(carga[m, p] * produtos[p]['valor']
         for m in mochilas
         for p in produtos)
)
modelo.optimize()
```

Listing 4. Função objetivo

3.4. Resolução e Interpretação dos Resultados

O solver utilizado resolve o problema por meio de programação linear inteira, utilizando técnicas como branch-and-bound para obter a solução ótima. Ao final, indica quais produtos devem ser levados (1) ou não (0).

```
for m in mochilas:
    produtos_mochila = [
        (p, produtos[p]['valor'], produtos[p]['peso'])
        for p in produtos if carga[m, p].x == 1
    ]
```

Listing 5. Leitura da solução

A biblioteca `pandas` pode ser utilizada para exibir os dados:

```
import pandas as pd
from tabulate import tabulate

df = pd.DataFrame.from_dict(produtos, orient='index')
print(tabulate(df, headers='keys', tablefmt='fancy_grid'))
```

Listing 6. Exibição dos dados

3.5. Visualização Gráfica

Para uma compreensão mais intuitiva da distribuição dos itens, utilizamos o pacote `matplotlib` para gerar gráficos de barras empilhadas.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.bar(nomes_mochilas, capacidades, color='gray', alpha=0.5,
    label='Capacidade Total')
plt.bar(nomes_mochilas, cargas_totais, color='blue', alpha=0.8,
    label='Carga Utilizada')

for i, v in enumerate(cargas_totais):
    plt.text(i, v + 50, f"{v}g", ha='center', fontsize=12)

plt.xlabel("Mochilas")
plt.ylabel("Peso (g)")
plt.title("Distribui o de Carga nas Mochilas")
plt.legend()
plt.grid(axis='y')
plt.show()
```

Listing 7. Gráfico de distribuição

3.6. Resultados e Discussão

A saída do programa mostra o valor total alcançado e a alocação de cada produto. Um exemplo de saída:

Tabela 6. Quantitativo de valores e pesos simulado para cada mochila.

Mochila	Valor	Peso
p_0	141	163
p_1	58	250
p_2	128	123
p_3	90	154
p_4	67	164
p_5	92	195
p_6	51	127
p_7	91	176
p_8	42	172
p_9	83	174
p_10	65	148
p_11	138	230
p_12	39	208
p_13	63	151
p_14	128	123

Tabela 7. Quantitativo de carga máxima gerada para cada mochila

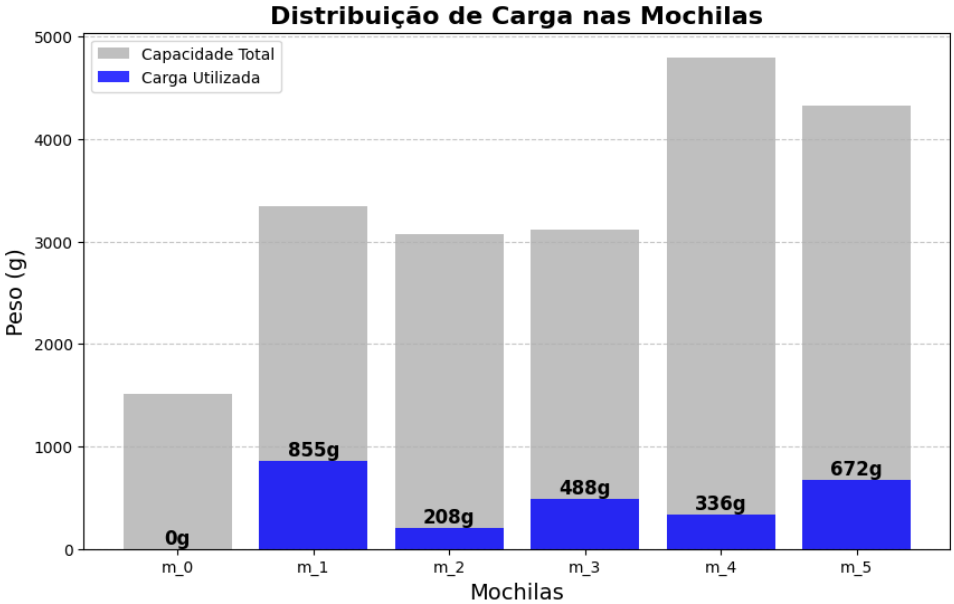
Mochila	Carga Maxima
m_0	1517
m_1	3339
m_2	3073
m_3	3117
m_4	4797
m_5	4320

VALOR TOTAL DAS MOCHILAS: R$ 1165.00

Tabela 8. Resumo geral dos valores simulados.

Mochila	Valor Total (R$)	Carga Total (g)	Espaço Livre (g)
m_0	0	0	1517
m_1	355	855	2484
m_2	39	208	2865
m_3	287	488	2629
m_4	109	336	4461
m_5	375	672	3648

Figura 1. distribuição dos itens simulados e alocados após a otimização númerica.



4. Resolução do Problema da Mochila com Algoritmo Guloso

Nesta etapa do trabalho, é apresentado o processo de definição do problema e a preparação dos dados para aplicação do algoritmo guloso. O problema consiste em distribuir itens, cada um com um valor e um peso, entre duas mochilas com capacidades limitadas. Para isso, os itens são representados em um dicionário que armazena suas características, enquanto as mochilas possuem suas capacidades máximas definidas.

A seguir, aplica-se o algoritmo guloso, que busca uma solução eficiente ao priorizar itens com a melhor razão valor/peso. Esses itens são ordenados de forma decrescente e, posteriormente, alocados sequencialmente nas mochilas conforme a disponibilidade de espaço. Essa abordagem visa maximizar o valor total transportado, respeitando as restrições de peso de cada mochila.

A solução desenvolvida apresentou uma alocação eficiente dos produtos entre as mochilas, maximizando o valor total transportado sem ultrapassar as capacidades definidas. O uso do MIP garantiu a obtenção de soluções otimizadas, e a visualização gráfica possibilitou uma análise clara da ocupação de cada mochila. Os resultados obtidos validam a abordagem adotada e servem como base para futuras extensões e melhorias no modelo.

4.1. Definição do Problema e Geração de Dados

Inicialmente, definem-se os itens, cada um caracterizado por um valor e um peso. Adicionalmente, duas mochilas são criadas, cada uma com uma capacidade máxima de carga. Estes dados são representados em estruturas do tipo dicionário no código:

```
itens = {
    "Item1": {"valor": 60, "peso": 10},
    "Item2": {"valor": 100, "peso": 20},
    "Item3": {"valor": 120, "peso": 30},
    "Item4": {"valor": 30, "peso": 5},
    "Item5": {"valor": 90, "peso": 15},
    "Item6": {"valor": 70, "peso": 25},
    "Item7": {"valor": 50, "peso": 10},
    "Item8": {"valor": 40, "peso": 8},
    "Item9": {"valor": 110, "peso": 18},
    "Item10": {"valor": 95, "peso": 12},
}
mochilas = {
    "Mochila1": 50,
    "Mochila2": 60
}
```

Listing 8. Definição dos Itens e Mochilas

Os itens são caracterizados por valores monetários e pesos em gramas, enquanto as mochilas possuem capacidades máximas também expressas em gramas.

4.2. Algoritmo Guloso para Seleção dos Itens

O Algoritmo Guloso é uma abordagem que realiza escolhas locais ótimas em cada etapa, com o objetivo de encontrar uma solução global. No contexto do problema, os itens são ordenados de acordo com a razão valor/peso em ordem decrescente, ou seja, aqueles que oferecem maior retorno por unidade de peso são priorizados.

A etapa de ordenação é executada da seguinte forma:

```
itens_ordenados = sorted(itens.items(), key=lambda x: x[1]['
    valor'] / x[1]['peso'], reverse=True)
```

Listing 9. Ordenação dos Itens pelo Algoritmo Guloso

A lista resultante permite que os itens de maior ”eficiência”sejam considerados primeiro para alocação.

4.3. Distribuição dos Itens entre as Mochilas

Após a ordenação, o algoritmo percorre a lista de itens e tenta alocá-los em uma das mochilas, verificando se ainda há capacidade disponível. Se o item couber, ele é adicionado à mochila e o espaço disponível é atualizado. Este processo continua até que todos os itens sejam processados.

```

mochilas_selecionadas = {m: [] for m in mochilas}
capacidades_restantes = mochilas.copy()

for item, info in itens_ordenados:
    for mochila in mochilas:
        if capacidades_restantes[mochila] >= info['peso']:
            mochilas_selecionadas[mochila].append((item, info['
                valor'], info['peso']))
            capacidades_restantes[mochila] -= info['peso']
            break
```

Listing 10. Distribuição dos Itens entre as Mochilas

Cada item é tentado nas mochilas em sequência. Caso caiba, ele é armazenado e o processo continua. Caso contrário, o item é ignorado e o próximo é testado.

4.4. Apresentação dos Resultados

Ao final da execução do algoritmo, os itens alocados em cada mochila são listados, junto com seus valores, pesos, o total acumulado e o espaço restante disponível. A visualização é organizada de forma clara para interpretação dos resultados.

```python
print("\nDistribui o dos itens nas mochilas (Algoritmo Guloso):")
for mochila, itens_mochila in mochilas_selecionadas.items():
    peso_total = sum(peso for _, _, peso in itens_mochila)
    valor_total = sum(valor for _, valor, _ in itens_mochila)
    capacidade_restante = mochilas[mochila] - peso_total

    print(f"\n {mochila} (Capacidade: {mochilas[mochila]}g)")
    for item, valor, peso in itens_mochila:
        print(f"{item}: Valor = R$ {valor}, Peso = {peso}g")
    print(f"\n Valor Total: R$ {valor_total},00 |  Carga Total: {peso_total}g |  Ociosidade: {capacidade_restante}g")
    print("="*40)
```

4.4.1. Distribuição dos itens nas mochilas (Algoritmo Guloso)

Tabela 9. Itens alocados na Mochila 1 (Capacidade: 50g) após a otimização.

Item	Valor (R$)	Peso (g)
Item0	95	12
Item9	110	18
Item1	60	10
Item4	30	5
Valor Total: R$ 295,00		
Carga Total: 45g		
Ociosidade: 5g		

Tabela 10. Itens alocados na Mochila 2 (Capacidade: 60g) após a otimização.

Item	Valor (R$)	Peso (g)
Item5	90	15
Item2	100	20
Item7	50	10
Item8	40	8
Valor Total: R$ 280,00		
Carga Total: 53g		
Ociosidade: 7g		

Tabela 11. Resumo das mochilas com os respectivos valores totais, cargas e espaços livres.

Mochila	Valor Total (R$)	Carga Total (g)	Espaço Livre (g)
Mochila 1	295	45	5
Mochila 2	280	53	7

O algoritmo guloso mostrou-se eficiente para resolver o Problema da Mochila, maximizando o valor total transportado dentro das limitações de capacidade. A estratégia de priorizar itens com melhor relação valor/peso permitiu uma distribuição otimizada. Embora não garanta sempre a solução ótima global, é uma abordagem simples e eficaz para cenários que demandam rapidez e bom aproveitamento dos recursos

5. Comparação entre o Algoritmo Guloso e o Método Clássico na Solução do Problema da Mochila Múltipla

Para avaliar abordagens diferentes na resolução do Problema da Mochila Múltipla, foram implementadas duas soluções computacionais: o algoritmo guloso, baseado em uma lógica heurística, e o método clássico, que modela matematicamente o problema e busca a solução ótima.

Ambos os métodos foram aplicados sobre os mesmos dados gerados aleatoriamente, o que permite uma comparação justa entre as abordagens.

5.1. Preparação do Cenário: Geração dos Dados

A primeira etapa do código consiste na geração aleatória de dados para simular um cenário realista, onde há múltiplos itens e mochilas com capacidades distintas.

```
    import random
from tabulate import tabulate
from mip import Model, xsum, maximize, BINARY

# Definindo par metros principais
total_itens = 10
total_mochilas = 5
```

Aqui, definimos o número de itens disponíveis e o número de mochilas.

```
    # Gerar itens com valores e pesos aleat rios
itens = {
    f"Item{i+1}": {"valor": random.randint(10, 150),
                   "peso": random.randint(10, 3000)}
    for i in range(total_itens)
}
```

Cada item tem:

- Um valor entre R$ 10 e R$ 150
- Um peso entre 10g e 3000g

```
    # Gerar mochilas com capacidades aleat rias
mochilas = {
    f"Mochila{i+1}": random.randint(1000, 10000)
    for i in range(total_mochilas)
}
```

Cada mochila tem uma capacidade entre 1000g e 10000g.

Esses dados são exibidos em tabelas com a biblioteca tabulate para facilitar a visualização:

```
    # Tabela de itens
itens_tabela = [[item, f"R$ {info['valor']:,}", f"{info['peso
    ']:,} g"] for item, info in itens.items()]
print(tabulate(itens_tabela, headers=["Item", "Valor", "Peso"],
    tablefmt="fancy_grid"))

# Tabela de mochilas
mochilas_tabela = [[mochila, f"{capacidade:,} g"] for mochila,
    capacidade in mochilas.items()]
print(tabulate(mochilas_tabela, headers=["Mochila", "Capacidade"
    ], tablefmt="fancy_grid"))
```

5.2. Algoritmo Guloso

O algoritmo guloso é implementado com base em uma estratégia de decisão local ótima. Ele escolhe os itens com a melhor relação valor/peso.

```
    # Ordena  o dos itens pela eici ncia (valor/peso)
itens_ordenados = sorted(itens.items(), key=lambda x: x[1]['
    valor'] / x[1]['peso'], reverse=True)
```

Isso garante que os itens mais vantajosos (mais valor por grama) sejam escolhidos primeiro.

```
    mochilas_guloso = {m: [] for m in mochilas}
capacidade_restante = mochilas.copy()
```

Aqui criamos:

- Um dicionário para armazenar os itens em cada mochila.
- Uma cópia das capacidades para acompanhar o quanto ainda resta de espaço.

```
    for item, info in itens_ordenados:
    for mochila in mochilas:
        if capacidade_restante[mochila] >= info['peso']:
            mochilas_guloso[mochila].append((item, info['valor'
                ], info['peso']))
            capacidade_restante[mochila] -= info['peso']
            break  # Item alocado com sucesso
```

Para cada item, tentamos colocá-lo na primeira mochila que tiver espaço. Isso é feito em ordem de eficiência, o que acelera o processo e gera uma solução boa, mas nem sempre ideal.

5.3. Método Clássico (Modelo de Otimização)

Aqui usamos a biblioteca mip, que permite modelar o problema como uma otimização matemática.

```
    modelo = Model(sense=maximize)
```

O modelo é do tipo ”maximize”, ou seja, queremos maximizar o valor tota

```
    # Vari vel bin ria: 1 se o item i for colocado na mochila
       m
x = { (i, m): modelo.add_var(var_type=BINARY) for i in itens for
    m in mochilas }
```

Cada par item/mochila tem uma variável binária indicando se aquele item está ou não naquela mochila.

5.3.1. Restrições

Capacidade das mochilas:

```
    for m in mochilas:
    modelo += xsum(itens[i]['peso'] * x[i, m] for i in itens) <=
        mochilas[m]
```

Não se pode ultrapassar a capacidade de nenhuma mochila.

Item único por mochila:

```
    for i in itens:
    modelo += xsum(x[i, m] for m in mochilas) <= 1
```

5.3.2. Função Objetivo

```
    modelo.objective = maximize(xsum(itens[i]['valor'] * x[i, m]
        for i in itens for m in mochilas))
modelo.optimize()
```

Essa é a função que será maximizada: a soma dos valores dos itens alocados.

5.3.3. Reconstrução da Solução

```
    mochilas_pli = {m: [] for m in mochilas}
for i, m in x:
    if x[i, m].x >= 0.99:
        mochilas_pli[m].append((i, itens[i]['valor'], itens[i]['
            peso']))
```

Ao final, as decisões do solver são convertidas de volta em uma estrutura compreensível.

Tabela 12. Itens gerados com seus respectivos valores e pesos.

Item	Valor (R$)	Peso (g)
Item 1	133	1,614
Item 2	43	1,779
Item 3	47	1,830
Item 4	80	606
Item 5	121	1,531
Item 6	98	554
Item 7	55	2,122
Item 8	124	1,213
Item 9	144	2,908
Item 10	102	2,002

Tabela 13. Capacidades de cada mochila.

Mochila	Capacidade (g)
Mochila 1	2,965
Mochila 2	7,227
Mochila 3	4,670
Mochila 4	1,654
Mochila 5	7,760

Tabela 14. Distribuição dos itens nas mochilas utilizando o algoritmo guloso.

Mochila	Item	Valor (R$)	Peso (g)
Mochila 1	Item 6	98	554
Mochila 1	Item 4	80	606
Mochila 1	Item 8	124	1,213
Mochila 2	Item 1	133	1,614
Mochila 2	Item 5	121	1,531
Mochila 2	Item 10	102	2,002
Mochila 2	Item 3	47	1,830
Mochila 3	Item 9	144	2,908
Mochila 5	Item 7	55	2,122
Mochila 5	Item 2	43	1,779

Tabela 15. Distribuição dos itens nas mochilas utilizando o método clássico de otimização.

Mochila	Item	Valor (R$)	Peso (g)
Mochila 1	Item 4	80	606
Mochila 1	Item 6	98	554
Mochila 2	Item 7	55	2,122
Mochila 2	Item 9	144	2,908
Mochila 3	Item 2	43	1,779
Mochila 4	Item 5	121	1,531
Mochila 5	Item 1	133	1,614
Mochila 5	Item 3	47	1,830
Mochila 5	Item 8	124	1,213
Mochila 5	Item 10	102	2,002

Tabela 16. Comparativo das abordagens Guloso e Otimização Clássica quanto ao valor total, peso total e eficiência.

Abordagem	Valor Total (R$)	Peso Total (g)	Eficiência (Valor/Peso)
Guloso	947	16,159	0,0586
Otimização Clássica	947	16,159	0,0586

5.4. Considerações Finais

Este experimento mostrou de forma clara as diferenças entre duas abordagens para o mesmo problema:

- O algoritmo guloso se destaca pela rapidez e simplicidade. Ele é indicado quando se precisa de uma solução rápida, mesmo que ela não seja a melhor. É útil em sistemas embarcados, aplicações móveis e contextos de tempo real.
- O método clássico garante a melhor solução possível, considerando todas as combinações viáveis. Ele é mais lento, mas essencial em contextos que exigem precisão máxima, como planejamento logístico, controle de estoques e transporte de cargas valiosas.

Embora ambos os métodos tenham retornado o mesmo valor total neste caso, as alocações distintas reforçam a importância de compreender o comportamento de cada abordagem. O algoritmo guloso pode funcionar bem — mas confiar cegamente nele pode levar a decisões subótimas em instâncias mais complexas.

Referências

MARTELLO, S.; TOTH, P. *Knapsack Problems: Algorithms and Computer Implementations*. [S.l.]: John Wiley & Sons, 1990.